TP1 – Suivi et Planification de Trajectoire d'un robot mobile à roues différentielles

Partie 1 : Modélisation, Commande et Planification (A*)
Partie 2 : Localisation par distance aux murs (jumeau numérique)

Julien Dubuc

Table des matières

Introduction **3**

I Modélisation, Commande et Planification **3**

1 Modèle et contrôle du robot **3**
1.1 Gain statique du moteur . . . 3
1.2 Régulateur d'un moteur . . . 3
1.3 Modèle cinématique du robot . . . 4
1.4 Modèle Simulink en boucle ouverte et validation . . . 4
1.5 Commande bas niveau en boucle fermée . . . 4

2 Planification de trajectoire **5**
2.1 Milieu dégagé : atteindre un point B . . . 5
2.2 Suivi d'une succession de points B_i . . . 6
2.3 Planification en présence d'obstacles : algorithme A* . . . 7

II Localisation par Distance aux Murs (Jumeau Numérique) **9**

3 Objectif et principe **9**

4 Rappel théorique : mesure de distance à un mur **9**
4.1 Filtre de Kalman . . . 10

5 Mise en œuvre dans le projet **10**
5.1 Construction de la scène (`planification_astar.m`) . . . 10
5.2 Planificateur à mémoire (`planificateur.m`) . . . 10
5.3 Capteur virtuel et filtre de Kalman (`estimateur_murs.m`) . . . 10
5.4 Correspondance avec le cours . . . 11

6 Intégration dans le modèle Simulink **11**

7 Résultats **11**
7.1 Trajectoire . . . 11
7.2 Position estimée . . . 12
7.3 Vitesse . . . 13
7.4 Cap . . . 13

8 Limites et améliorations **14**

Conclusion **14**

Introduction

Ce TP porte sur le contrôle d'un robot à deux roues motrices indépendantes (roue gauche, roue droite). Les caractéristiques physiques sont : écartement des roues $L = 20$ cm $= 0{,}2$ m, diamètre des roues $D = 40$ mm soit un rayon $R = 0{,}02$ m, et tensions d'alimentation des moteurs $u_g, u_d \in [-5\,\mathrm{V}, +5\,\mathrm{V}]$.

La **partie 1** traite le sujet du TP : modèle du robot en boucle ouverte, commande bas niveau en boucle fermée (vitesse v et cap θ), puis planification de trajectoire en milieu dégagé et en présence d'obstacles (algorithme A*). La **partie 2** ajoute une couche de perception : le robot ne connaît plus sa position, mais seulement des *distances bruitées aux murs*, à partir desquelles un filtre de Kalman estime sa position ; le planificateur pilote alors à partir de cette estimation.

Partie I

Modélisation, Commande et Planification

1 Modèle et contrôle du robot

1.1 Gain statique du moteur

D'après l'énoncé (RISE, *Contrôle non-linéaire appliquée*), la relation entre la tension u_d (resp. u_g) et la vitesse de rotation ω_d (resp. ω_g) de la roue est

$$H(s) = \frac{K}{1 + \dfrac{s}{10} + \dfrac{s^2}{2000}},$$

où K est le gain du système (modèle du second ordre). En régime permanent ($s = 0$) on a $\omega = K\,u$. À pleine puissance, $u_{max} = 5$ V et le moteur tourne à 180 tr/min :

$$\omega_{max} = 180 \times \frac{2\pi}{60} = 6\pi \approx 18{,}85\ \mathrm{rad/s}, \qquad K = \frac{\omega_{max}}{u_{max}} = \frac{18{,}85}{5} \approx 3{,}77\ \mathrm{rad/s/V}\ (= 36\ \mathrm{tr/min/V}).$$

Dans le modèle Simulink, chaque moteur est représenté par la fonction de transfert

$$H(s) = \frac{3{,}77}{\dfrac{1}{2000}s^2 + \dfrac{1}{10}s + 1},$$

soit exactement la forme de l'énoncé. Sa dynamique est dominée par une constante de temps d'environ 0,1 s (pôle lent vers -10 rad/s, pôle rapide vers -200 rad/s).

1.2 Régulateur d'un moteur

L'énoncé laisse libres les performances attendues. Le moteur étant dominé par un pôle lent, un correcteur PI $C(s) = K_p\left(1 + \frac{1}{T_i s}\right)$ (ou un P/PID) permet d'annuler l'erreur statique. Comme la consigne de tension est physiquement bornée à ± 5 V, le bloc PID de Simulink est configuré avec une **saturation** de -5 à $+5$ et un mécanisme **anti-windup** (mode *clamping*) afin que l'intégrateur ne s'emballe pas quand la commande sature.

1.3 Modèle cinématique du robot

La vitesse linéaire et la vitesse de rotation dépendent des vitesses angulaires des roues ω_d, ω_g :

$$v_d = R\,\omega_d, \qquad v_g = R\,\omega_g, \tag{1}$$

$$v = \frac{v_d + v_g}{2} = \frac{R}{2}\,(\omega_d + \omega_g), \tag{2}$$

$$\dot{\theta} = \frac{v_d - v_g}{L} = \frac{R}{L}\,(\omega_d - \omega_g), \tag{3}$$

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta. \tag{4}$$

Avec $\omega_d = H(s)u_d$ et $\omega_g = H(s)u_g$, les tensions de commande sont reliées à $(v, \dot{\theta})$. Dans Simulink, les gains valent $R/2 = 0{,}02/2$ pour v et $R/L = 0{,}02/0{,}2$ pour $\dot{\theta}$.

1.4 Modèle Simulink en boucle ouverte et validation

Le modèle comporte deux blocs $H(s)$ (un par moteur) précédés d'une saturation ± 5 V, deux sommateurs et deux gains pour obtenir v et $\dot{\theta}$, puis des intégrateurs pour θ, et pour x et y après multiplication par $\cos\theta$ et $\sin\theta$. Trois essais de validation en boucle ouverte sont prévus :

— **ligne droite** : $u_d = u_g = 3$ V ;
— **cercle** : tensions constantes différentes, $u_d = 4$ V, $u_g = 2$ V ;
— **spirale** : $u_d = 5$ V constant et u_g décroissant en rampe de 5 à 1 V.

1.5 Commande bas niveau en boucle fermée

Il est difficile de réguler v par un PID agissant sur un seul moteur, puisque v dépend des deux. On utilise donc une structure découplée : v est régulé en agissant de façon *identique* sur les deux moteurs, et le cap θ en agissant sur la *différence* de tensions.

1. Un PID compare v^* à v et fournit la commande commune U_v.
2. Un PID compare θ^* à θ et fournit la commande différentielle U_θ.
3. Découplage : $u_d = U_v + U_\theta$ et $u_g = U_v - U_\theta$ (dans le modèle, un gain de 0,5 est appliqué à la somme).
4. Un régulateur interne par moteur asservit ω_d et ω_g.

Tous les PID sont saturés à ± 5 V avec anti-windup (*clamping*).

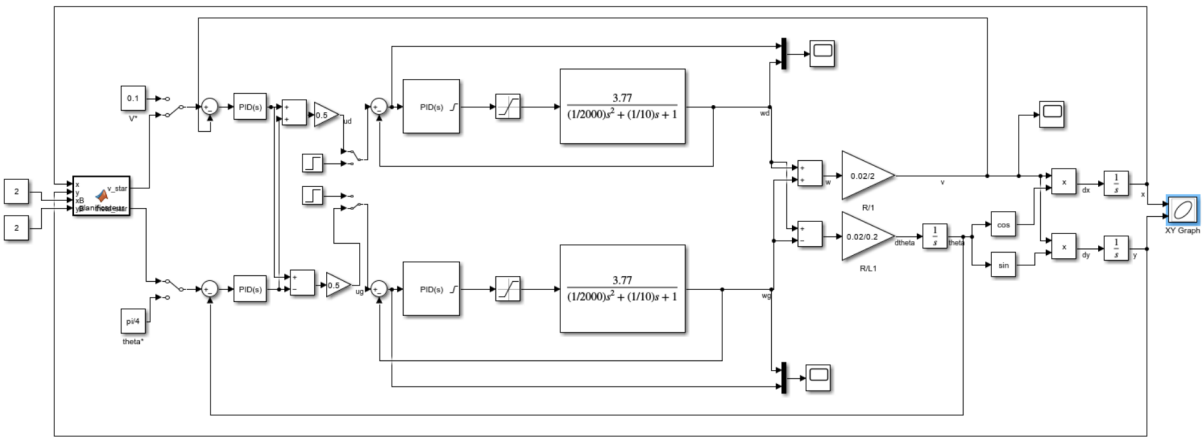


Figure 1 – Modèle Simulink de la commande en cascade (vitesse et cap), moteurs $H(s)$ saturés, cinématique unicycle.

Test. Pour rouler à 10 cm/s dans la direction $x = y$ on fixe $v^* = 0{,}1$ m/s et $\theta^* = \pi/4$ (45°). La figure 2 montre la trajectoire obtenue.

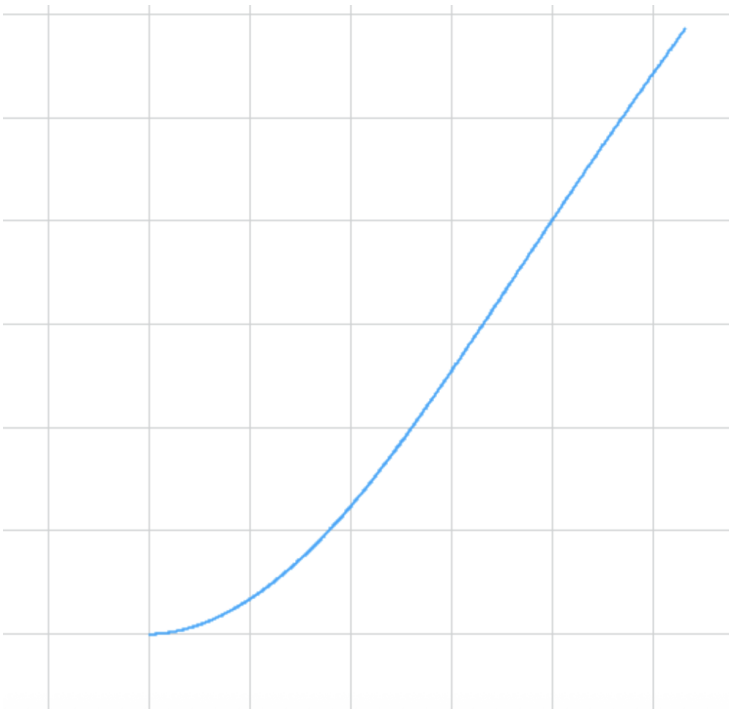


Figure 2 – Trajectoire (x, y) pour $v^* = 0{,}1$ m/s et $\theta^* = \pi/4$.

Analyse. La trajectoire se stabilise en une droite diagonale : le robot a atteint et maintient le cap de $\pi/4$ (direction $x = y$). La courbure initiale s'explique par un cap initial nul (robot aligné sur l'axe des abscisses) : un véhicule réel ne pivote pas instantanément à 45°, et cette courbe correspond au temps de réponse de la boucle de cap, pendant lequel le robot avance tout en tournant.

2 Planification de trajectoire

2.1 Milieu dégagé : atteindre un point B

Le robot connaît sa position en temps réel (LIDAR, géolocalisation…). Pour atteindre un point $B = (x_B, y_B)$:

$$\theta^* = \operatorname{atan2}(y_B - y,\ x_B - x), \tag{5}$$

$$d = \sqrt{(x_B - x)^2 + (y_B - y)^2}, \tag{6}$$

$$v^* = \operatorname{sat}(K_v\, d), \tag{7}$$

avec $K_v = 0{,}5$, une saturation haute à 0,5 m/s (plafond), une saturation basse à 0,1 m/s (plancher, pour éviter l'arrêt prématuré près de la cible) et un arrêt total ($v^* = 0$, $\theta^* = 0$) quand $d < 0{,}05$ m.

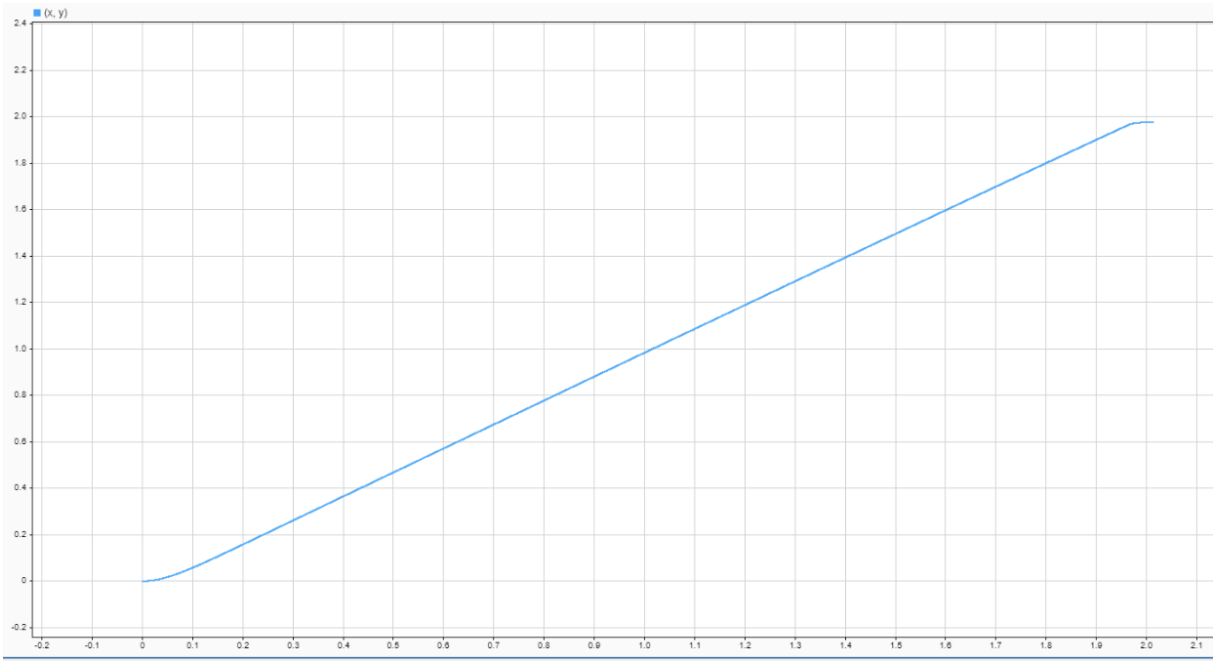


Figure 3 – Trajectoire vers $B = (2, 2)$: ligne droite de $(0, 0)$ à $(2, 2)$ avec arrêt final.

Le robot rejoint le point $(2; 2)$ en ligne droite, après un bref transitoire de rotation en début de trajet, puis s'arrête.

2.2 Suivi d'une succession de points B_i

Pour suivre une liste de points de passage $(B_i)_{i=1..N}$, le planificateur reçoit les vecteurs complets `chemins_X` et `chemins_Y` et gagne une *mémoire* :

— une variable `persistent index_cible` stocke le point visé ;
— quand $d < 0{,}05$ m, l'indice est incrémenté pour viser B_{i+1} ;
— le robot ne coupe ses moteurs ($v^* = 0$) qu'au dernier point B_N.

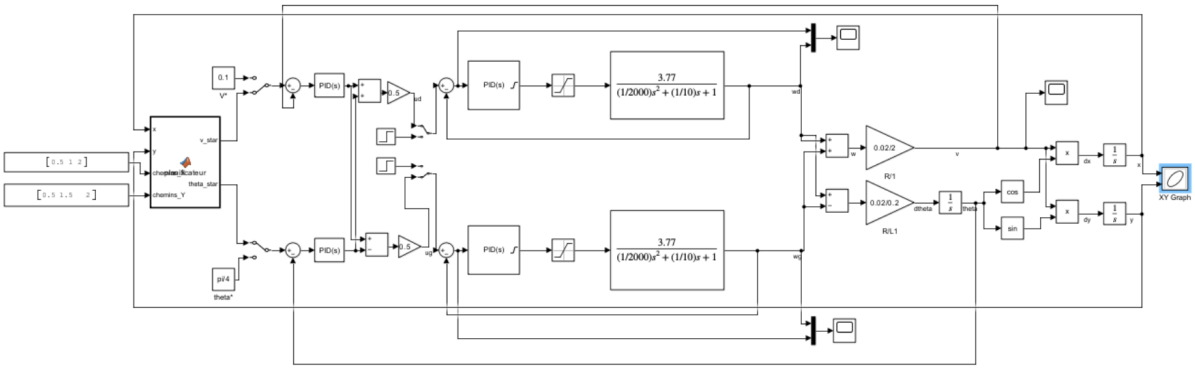


Figure 4 – Modèle Simulink avec la liste de points [0,5 1 2] en x et [0,5 1,5 2] en y.

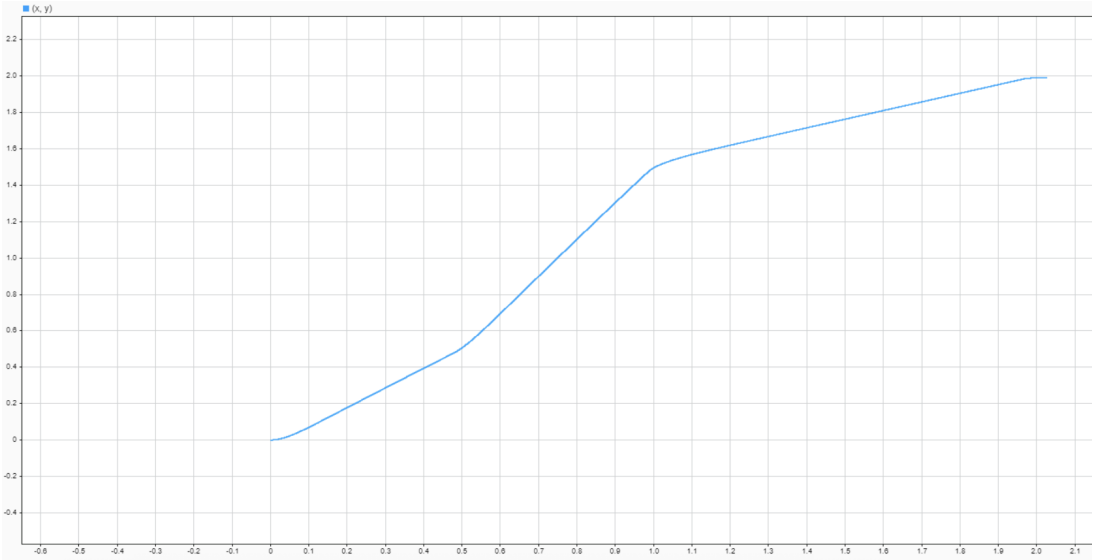


Figure 5 – Trajectoire passant par $(0{,}5; 0{,}5)$, $(1; 1{,}5)$ puis $(2; 2)$: le robot change de direction à chaque point de passage.

2.3 Planification en présence d'obstacles : algorithme A*

Principe. L'espace est discrétisé en une grille : chaque cellule libre est un nœud praticable, les obstacles sont des nœuds bloqués. A* évalue pour chaque nœud n le coût

$$f(n) = g(n) + h(n),$$

où $g(n)$ est le coût réel du départ jusqu'à n et $h(n)$ une heuristique, ici la distance euclidienne à vol d'oiseau jusqu'à l'arrivée. Le résultat est une liste de points $(B_i)_{i=0..N}$ contournant les obstacles, calculée *hors ligne* puis envoyée au planificateur de la section précédente.

Mise en œuvre (workflow).

1. **Environnement** : une matrice du workspace avec 0 (libre) et 1 (obstacle) ; ici une arène de 20×20 cases de 0,1 m.
2. **A*** : exploration à 8 voisins (diagonales de coût $\sqrt{2}$), listes *ouvert* et *fermé*, matrices de parents pour la reconstruction du chemin, interdiction de couper le coin d'un obstacle.
3. **Filtrage** : le chemin brut (case par case) est réduit aux seuls points de virage, en comparant les vecteurs de déplacement successifs. Cela évite d'asphyxier le robot avec des points trop rapprochés (oscillations, spirales dans Simulink).
4. **Mise à l'échelle** : les indices de grille sont convertis en mètres via la résolution $res = 0{,}1$ (ex. la case $(2, 2)$ devient $(0{,}2; 0{,}2)$ m et $(18, 18)$ devient $(1{,}8; 1{,}8)$ m), car le modèle Simulink manipule des distances physiques.
5. **Export** : `chemins_X_workspace` et `chemins_Y_workspace` alimentent directement le bloc planificateur.

Améliorations apportées. Carte plus riche (ex. 30×30 avec plusieurs obstacles, figure 6), marge de sécurité par gonflage des obstacles (`conv2` avec noyau 3×3), filtrage des points d'inflexion, et commande en cascade avec anti-windup.

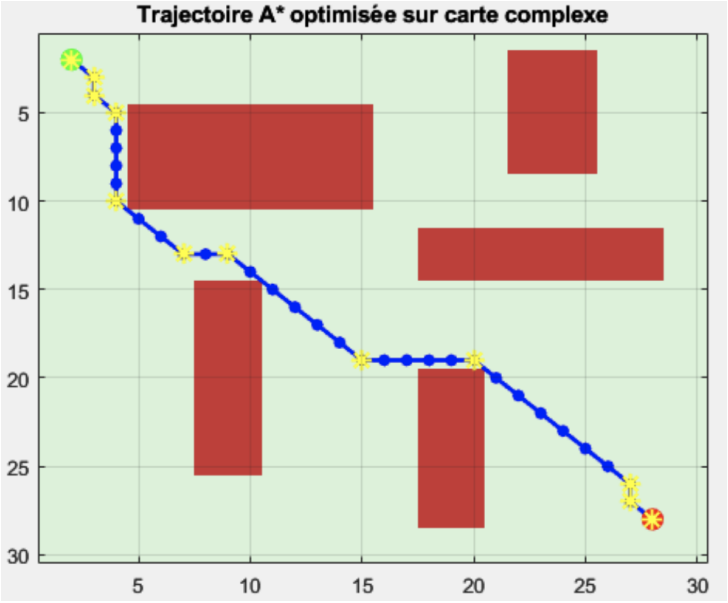


Figure 6 – Trajectoire A* optimisée sur une carte complexe (départ vert, arrivée rouge)

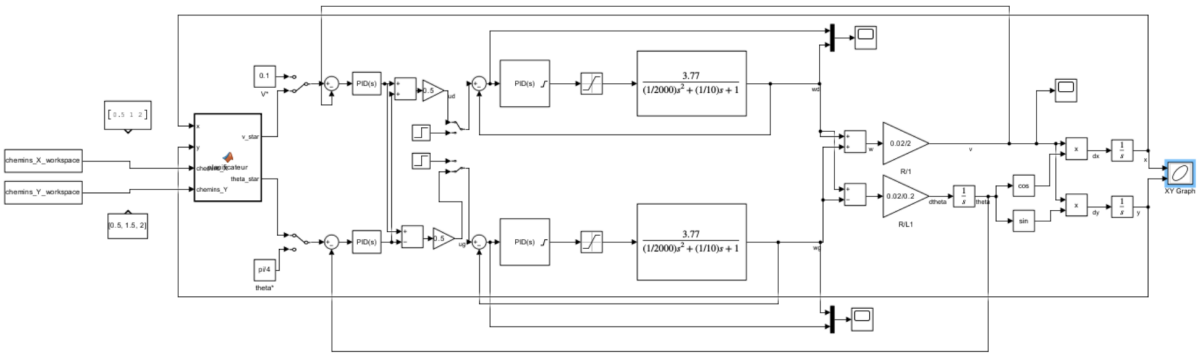


Figure 7 – Modèle Simulink alimenté par `chemins_X_workspace` et `chemins_Y_workspace`.

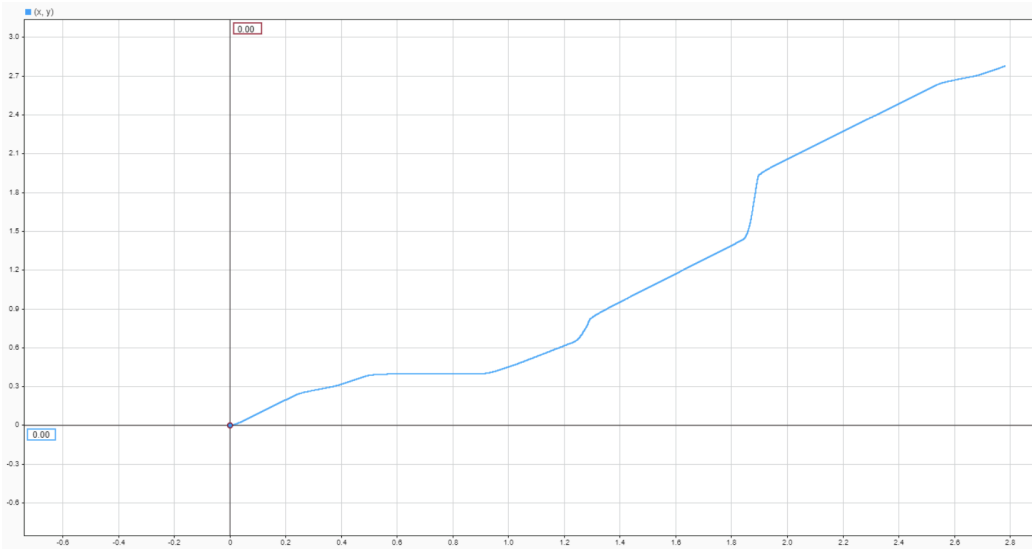


Figure 8 – Trajectoire simulée suivant les waypoints A*

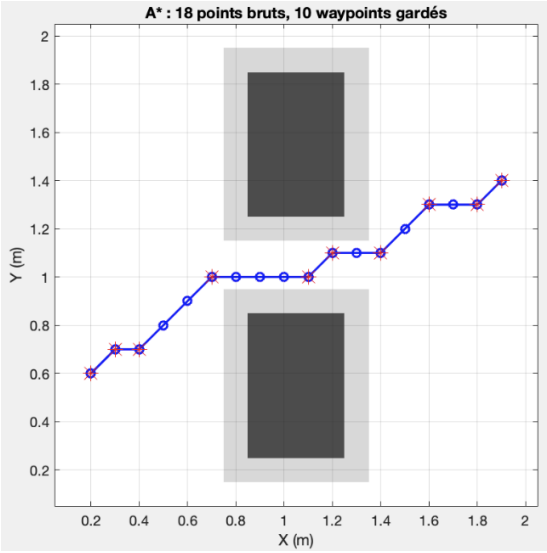


Figure 9 – Arène 2×2 m avec deux obstacles : 18 points bruts, 10 waypoints conservés après filtrage.

Partie II

Localisation par Distance aux Murs (Jumeau Numérique)

3 Objectif et principe

Dans la partie 1, la position (x, y) est supposée connue. L'énoncé précise en note que le robot pourrait se géolocaliser à l'aide d'un LIDAR, ce qui « nécessiterait d'utiliser un observateur » : c'est l'objet de cette partie. Ici, le modèle Simulink devient un **jumeau numérique** : un filtre de Kalman, qui ne voit que des *distances bruitées aux murs*, estime la position du robot, et le planificateur pilote vers l'objectif à partir de **cette estimation**. La position vraie ne sert qu'à fabriquer les mesures du capteur virtuel.

4 Rappel théorique : mesure de distance à un mur

Un mur est un segment $[a, b]$. Avec $u = \dfrac{b-a}{\|b-a\|}$, la distance signée d'un point P à la droite du mur est

$$d = \det(u,\, P - a) = u_1(P_2 - a_2) - u_2(P_1 - a_1).$$

Les murs sont orientés de sorte que l'espace libre soit *à gauche* ($d > 0$ côté robot). Cette mesure est rendue **linéaire** en la position $x = (x_1, x_2)^T$:

$$y_i = d_i + u_1 a_2 - u_2 a_1 = -u_2\, x_1 + u_1\, x_2, \qquad C_i = \begin{pmatrix} -u_2 & u_1 \end{pmatrix}.$$

4.1 Filtre de Kalman

Pour un état x de covariance Γ, bruit de mesure Γ_β :

$$S = C\,\Gamma\,C^T + \Gamma_\beta, \tag{8}$$
$$K = \Gamma\,C^T S^{-1}, \tag{9}$$
$$\hat{x} \leftarrow \hat{x} + K\,(y - C\hat{x}), \tag{10}$$
$$\Gamma \leftarrow \Gamma - K\,C\,\Gamma. \tag{11}$$

Dans le cours (cas statique, $A = I$), il n'y a pas d'étape de prédiction. Dans le cas dynamique, on ajoute $\hat{x} \leftarrow A\hat{x}$ et $\Gamma \leftarrow A\Gamma A^T + \Gamma_\alpha$.

5 Mise en œuvre dans le projet

5.1 Construction de la scène (`planification_astar.m`)

Le script définit une arène de 2×2 m (grille 20×20 de 0,1 m) avec deux obstacles rectangulaires centraux laissant un passage entre eux, une case de départ à gauche et une d'arrivée à droite. Il gonfle les obstacles d'une case (marge de sécurité), lance A* (8 voisins, diagonales $\sqrt{2}$, heuristique euclidienne, pas de coupe de coin), filtre le chemin pour ne garder que les waypoints, et exporte `chemins_X_workspace` et `chemins_Y_workspace`. Il construit aussi `murs`, matrice de 12 segments (4 bords de l'arène et 4 côtés par obstacle) :

C'est la « vérité terrain » géométrique du jumeau : la liste des waypoints (entrée du planificateur) et la liste des murs (entrée de l'estimateur).

5.2 Planificateur à mémoire (`planificateur.m`)

Version finale du planificateur : variables persistantes pour l'indice du waypoint, le dernier cap et un drapeau `fini`. Un waypoint est atteint à moins de 0,10 m ; la vitesse est $v^* = \min(0{,}5\,d,\ 0{,}3)$; après le dernier waypoint, $v^* = 0$ et θ^* conserve le dernier cap pour éviter un demi-tour sur place.

5.3 Capteur virtuel et filtre de Kalman (`estimateur_murs.m`)

Le bloc `estimateur_murs` contient deux parties.

1) Capteur virtuel. Pour chaque mur : vecteur unitaire u, distance signée $d = \det(u, P - a)$ et position t du pied de la perpendiculaire. Un mur est « vu » ($w = 1$) si $0 \le t \le L$, $d \ge 0$ et $d \le 1{,}2$ m (portée). On ajoute un bruit gaussien d'écart-type $\sigma = 3$ cm.

2) Filtre de Kalman. État $X = [x; v_x; y; v_y]^T$, initialisé *volontairement faux* (position décalée de 40 cm, vitesse nulle, $\Gamma = \mathrm{diag}(1, 0{,}1, 1, 0{,}1)$). Prédiction à vitesse constante avec $\Delta t = 0{,}1$ s :

$$A = \begin{pmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad Q_b = q \begin{pmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{pmatrix}, \quad q = Q_{ACC} = 0{,}05,$$

avec $\Gamma_\alpha = \mathrm{diag}(Q_b, Q_b)$. La mesure linéarisée est $y_{lin,i} = w_i\,(d_i + u_{1,i}a_{2,i} - u_{2,i}a_{1,i})$, avec $C_i = [-w_i u_{2,i},\ 0,\ w_i u_{1,i},\ 0]$. Les murs non vus ont une ligne nulle dans C et un y_{lin} nul : la dimension de mesure reste constante (pratique pour Simulink).

5.4 Correspondance avec le cours

	Cours (statique)	Estimateur (dynamique)
Mesure linéarisée	$y_i = d_i + u_1 a_2 - u_2 a_1$, ligne $[-u_2,\ u_1]$	identique, avec deux zéros pour v_x, v_y et facteur w
Bruit	$\Gamma_\beta = I_3$	$\sigma^2 I_{N_m}$, $\sigma = 3$ cm
Mise à jour	$S, K, \hat{x}, \Gamma$	identique + symétrisation de Γ
Prédiction	aucune ($A = I$)	$X \leftarrow AX$, $\Gamma \leftarrow A\Gamma A^T + \Gamma_\alpha$
Distances	données	fabriquées par le capteur virtuel

Table 1 – Estimateur implémenté et formules du cours.

Le facteur w vaut 1 pour un mur vu (on retombe exactement sur la formule du cours) et 0 sinon (ligne nulle, sans effet sur le calcul).

6 Intégration dans le modèle Simulink

Le modèle d'origine (PID en cascade de vitesse et de cap, deux moteurs, cinématique unicycle) est inchangé. L'estimateur se place entre la sortie de position du robot et le planificateur :

- x et y vrais passent par deux *Zero-Order Hold* (0,1 s) vers `estimateur_murs`, avec un bloc `Constant` contenant `murs` ;
- x_{est} et y_{est} **remplacent** x et y à l'entrée du planificateur ;
- les intégrateurs de x et y démarrent au premier waypoint (0,2 m, 0,6 m) : à l'origine, hors de l'arène, aucun mur n'est visible et l'estimation resterait figée ;
- le PID de vitesse a été réaccordé pour que v suive v^* ;
- des scopes affichent x, x_{est}, y, y_{est}, v et v^*, θ et θ^*, et `nb_murs`.

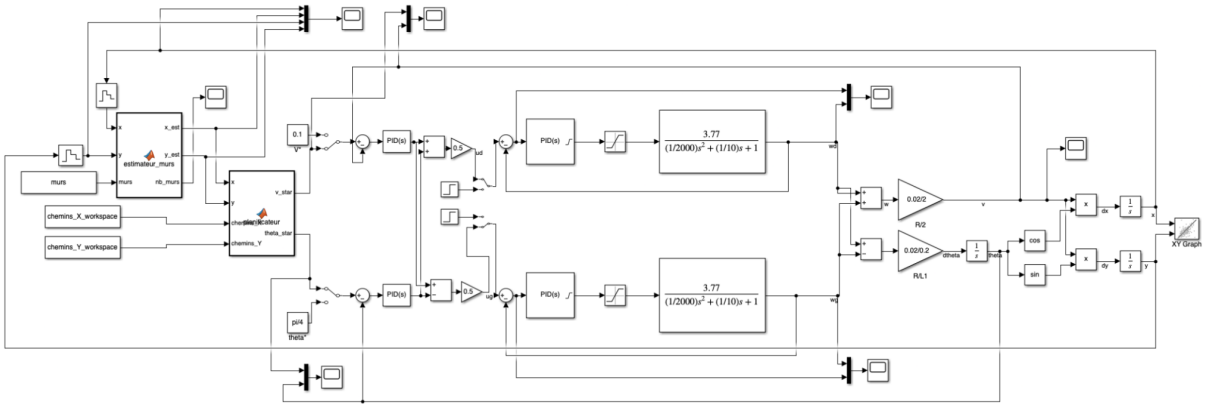


Figure 10 – Modèle Simulink complet du jumeau numérique (`TP1_DISTANCEMURS`).

7 Résultats

7.1 Trajectoire

Le robot atteint l'objectif (1,9 m; 1,4 m) en environ 17,5 s, en pilotant uniquement sur la position estimée.

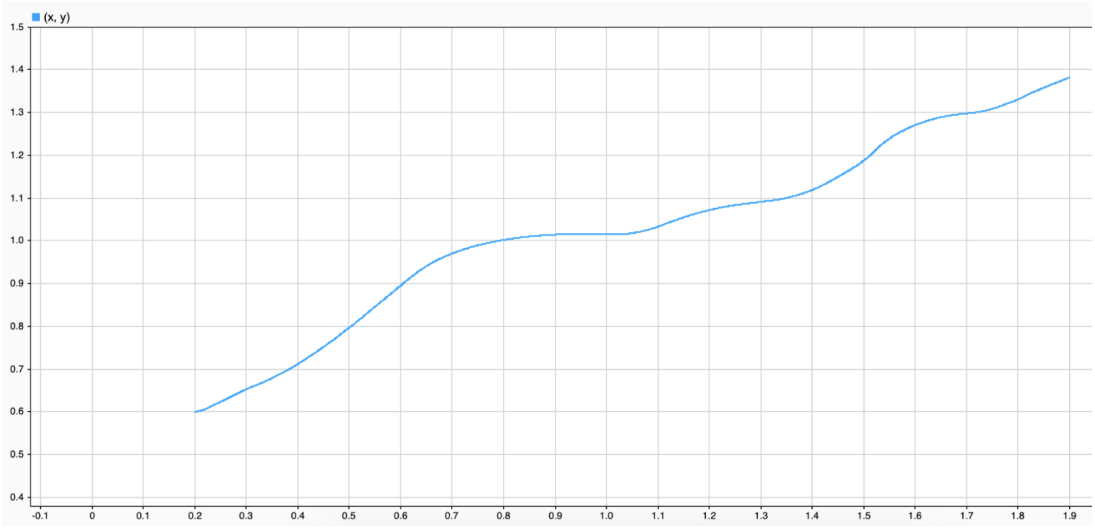


Figure 11 – Trajectoire (x, y) du robot (`TP1_DISTANCEMURS`) : passage entre les deux obstacles puis montée vers l'objectif.

7.2 Position estimée

x_{est} et y_{est} restent superposés aux x et y vrais pendant toute la course, dès $t = 0$, avec une dispersion d'environ 3 à 4 cm : c'est le bruit du capteur (3 cm), et il ne dérive pas. Entre $t = 7$ et 10 s, le robot traverse le passage entre les obstacles à $y \approx 1{,}0$–$1{,}1$ m. Le passage libre s'étend de 0,85 à 1,25 m : il franchit l'obstacle inférieur avec environ 17 cm de marge et le supérieur avec environ 23 cm.

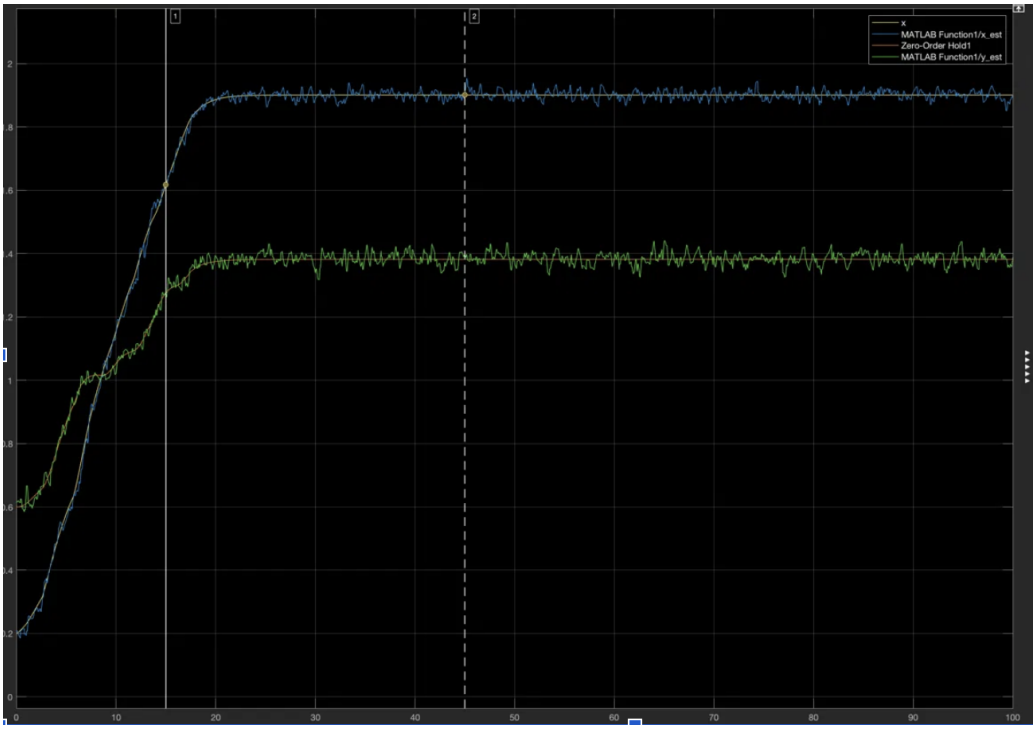


Figure 12 – x, x_{est}, y, y_{est} en fonction du temps : l'estimation converge dès le départ malgré l'initialisation fausse de 40 cm.

7.3 Vitesse

v suit v^* de près. La dent de scie de v^* vient du planificateur : v^* saute à 0,25 m/s à chaque changement de waypoint, puis diminue à mesure que le robot s'approche. Au dernier waypoint, v^* tombe à zéro et v s'éteint en quelques secondes.

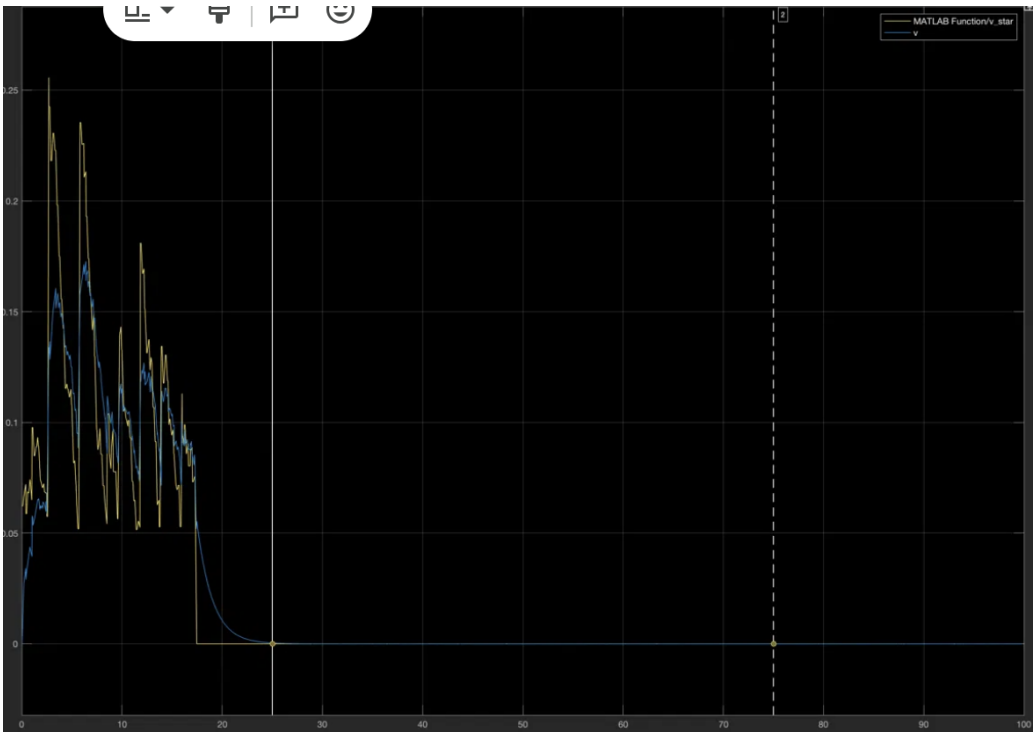


Figure 13 – Vitesse v et consigne v^*.

7.4 Cap

θ^* est bruité avant l'arrêt (entre $-0{,}15$ et 1,2 rad) : c'est un `atan2` du vecteur vers le waypoint, et lorsque ce vecteur est court, 3 cm de bruit de position font beaucoup varier l'angle. Le θ réel est lisse car la boucle de cap le filtre. Après l'arrêt, θ reste à 0,425 rad (24°) : le planificateur maintient son dernier cap, le robot ne tourne donc pas sur place.



Figure 14 – Cap θ et consigne θ^*.

8 Limites et améliorations

— L'estimation conserve une dispersion de 3 à 4 cm à l'arrêt, due au bruit de processus choisi ($Q_{ACC} = 0{,}05$) ; une valeur plus petite donne des courbes plus lisses, au prix d'une réactivité moindre.
— Le capteur voit les murs *à travers* les obstacles : l'occlusion n'est pas modélisée.
— Le bruit sur θ^* à courte distance d'un waypoint pourrait être réduit par un filtrage de θ^* ou une distance de commutation plus grande.

Conclusion

La partie 1 a permis de modéliser le robot à roues différentielles, de le commander en cascade (vitesse et cap, avec saturation et anti-windup), puis de lui faire suivre des points de passage issus d'un planificateur A*. La partie 2 a ajouté une localisation par distance aux murs : un filtre de Kalman à vitesse constante, alimenté par un capteur virtuel bruité, reconstruit la position à 3–4 cm près, et le robot rallie l'objectif en traversant le passage entre les obstacles uniquement à partir de sa position estimée.